

CS 423

Operating System Design:
Virtualization: CPU & Memory

Sep 08

Ram Alagappan

Logistics

MP0: Due on 09/14 11:59pm CT

4Cr: UNIX paper summary (was) due 2pm CT today

4Cr: Next paper on webpage; due by 09/15 2pm CT

This lecture:

- Finish up CPU virtualization

- Start memory virtualization - memory layout and translation

Next lecture:

- My phd student will teaching this one

AGENDA / LEARNING OUTCOMES

CPU virtualization

- Recap scheduling policies, finish MLFQ

- Proportional share scheduling

- Brief look at Linux scheduler

Memory virtualization

- What is the need for memory virtualization?

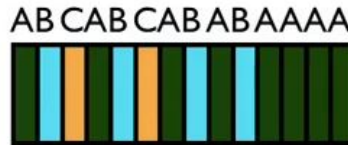
- Basics of virtualizing memory

RECAP: Scheduling Policies

Workload

JOB	arrival	run
A	0	40
B	0	20
C	5	10

Timelines



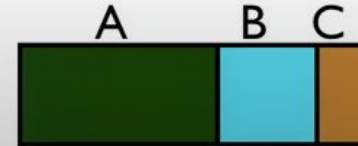
Schedulers:

FIFO

SJF

STCF

RR



MULTI-LEVEL FEEDBACK QUEUE

MLFQ: GENERAL PURPOSE SCHEDULER

Must support two job types with distinct goals

- “interactive” programs care about response time
- “batch” programs care about turnaround time

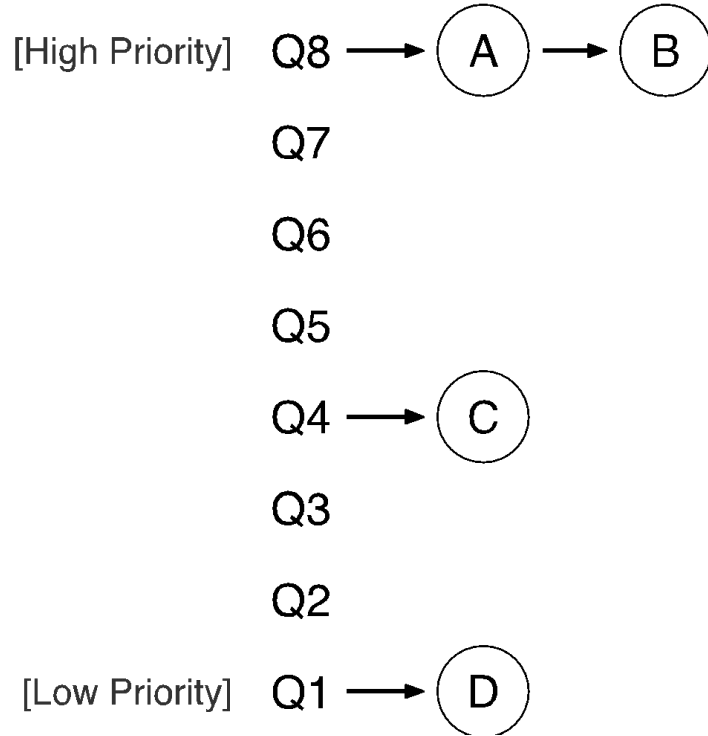
Approach:

Multiple levels of round-robin

Higher levels have higher priority

Processes are preemptable

MLFQ EXAMPLE



“Multi-level” – Each level is a queue

Rules for MLFQ

Rule 1: If $\text{priority}(A) > \text{Priority}(B)$
A runs

Rule 2: If $\text{priority}(A) == \text{Priority}(B)$,
A & B run in RR

CHALLENGES

How to set priority?

Does a process stay on one queue or move across queues?

Approach:

Use recent past behavior of process to predict future

Common approach in systems when don't have perfect knowledge

If interactive recently, will likely be interactive in the near future

MORE MLFQ RULES

Rule 1: If $\text{priority}(A) > \text{Priority}(B)$, A runs

Rule 2: If $\text{priority}(A) == \text{Priority}(B)$, A & B run in RR

Rule 3: Processes start at top priority

Rule 4: If job uses whole slice, demote process

(usually, longer time slices at lower priorities)

In other words, interactive jobs (that keep stopping for input) stay at the top priority.

ONE LONG JOB

Starts at top

Uses whole slice

Moves down

Moves down again

Q2

Q1

Q0

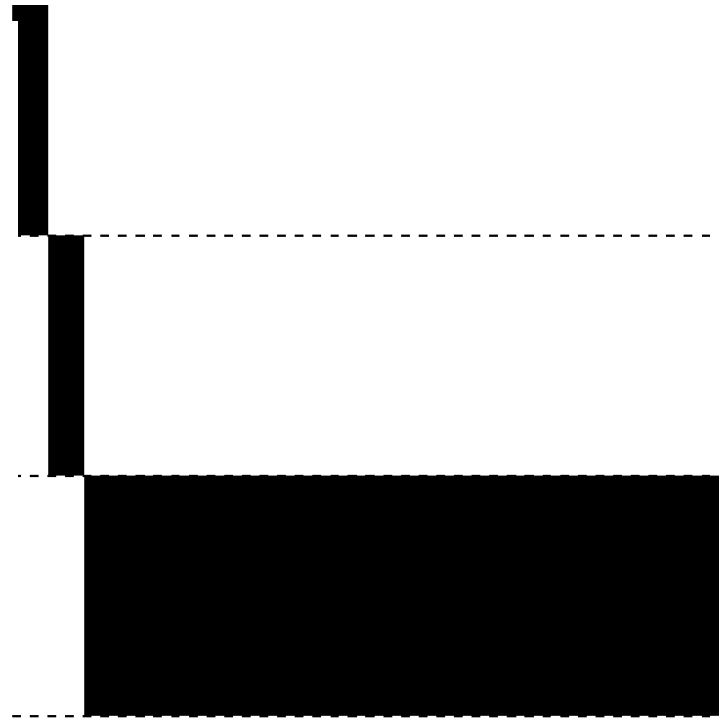
0

50

100

150

2



INTERACTIVE SHORT PROCESS JOINS



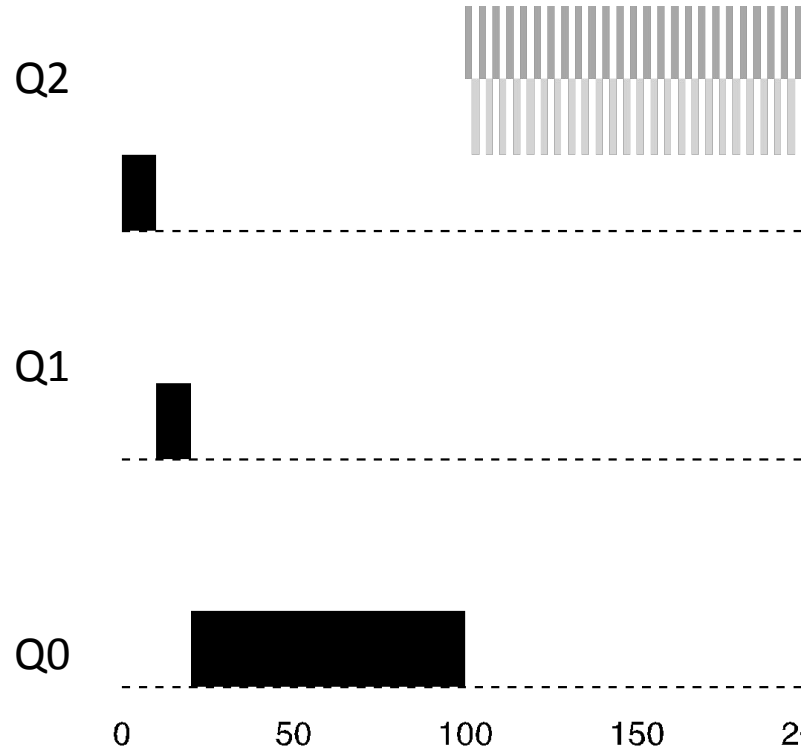
If new job is a short one

→ MLFQ mimics SJF

long-running interactive
job also gets good
response time

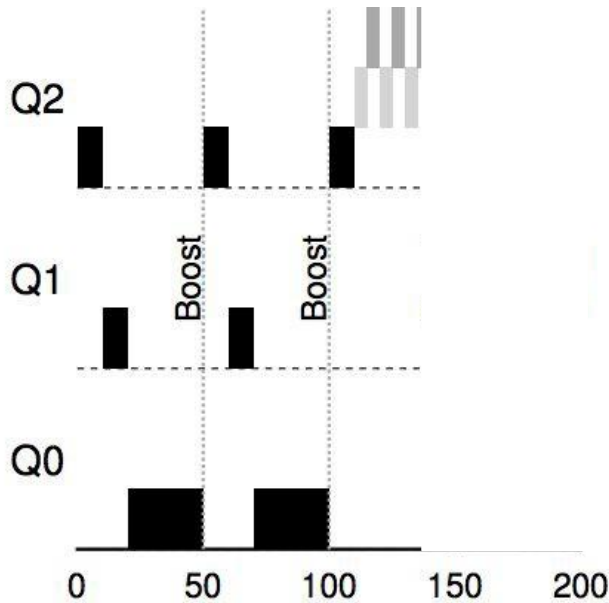
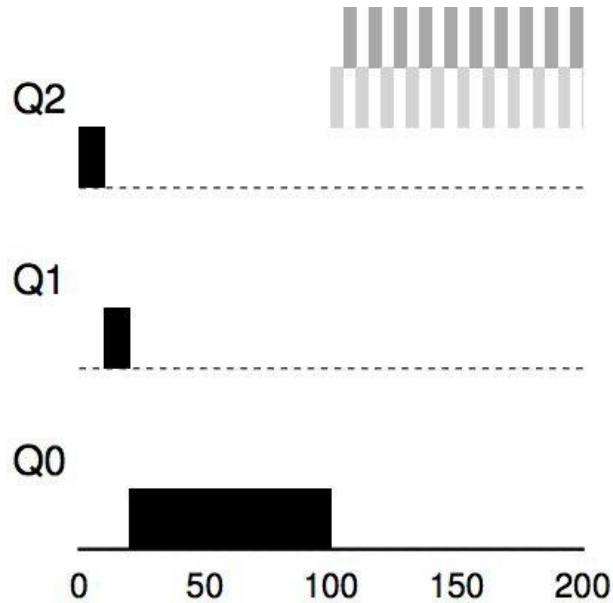
If it is a long running
batch job, it would move
down → optimize
turnaround

MLFQ PROBLEMS?



What is the problem with this schedule ?

AVOIDING STARVATION



Rule 5: After some time period S , move all the jobs in the system to the topmost queue.

Q: after the second boost, why aren't all processes run in RR?

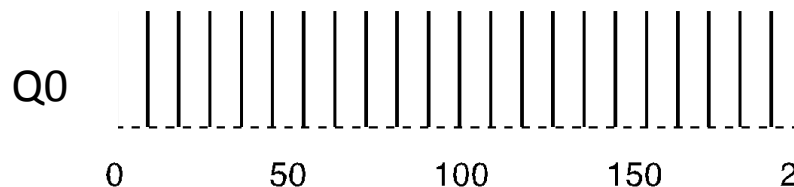
GAMING THE SCHEDULER ?



Job could trick scheduler by doing I/O just before time-slice end



When high-pri job is blocked, low pri job gets to run but when ready, it preempts the low-pri job



GAMING THE SCHEDULER ?

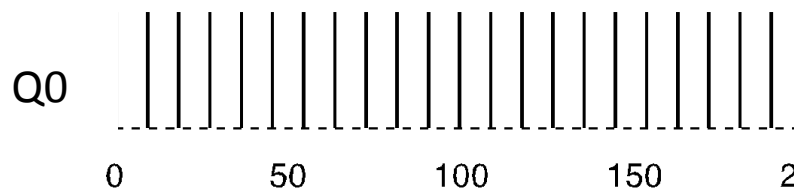


Job could trick scheduler by doing I/O just before time-slice end



Rule 4*: Account for total run time at priority level

Downgrade when exceed threshold



MLFQ Summary

Works well with a mix of compute-intensive and interactive jobs
Can finish shorter jobs earlier

5 rules to implement MLFQ

Versions of it are used in BSD, Sun Solaris, Windows NT, and Linux.

But multi-core versions!

Proportional Share in Scheduling

Metrics so far: turnaround time, response time

New metric: proportional-share

E.g., if Job A has paid 5x more price than Job B, then A is 5 times more likely to get the CPU than B

i.e., A is 5x higher pri than B

If both paid equally, A and B are equally likely to get the CPU

Lottery Scheduling: Probabilistically Proportional

Give each job tickets; #tickets based on priority

Conduct a lottery every period

Winner gets scheduled on CPU

Lottery: Generate a random number $0 \leq r < 60$; schedule:

A if $0 \leq r < 10$

B if $10 \leq r < 30$

C if $30 \leq r < 60$

A	10 tickets
B	20 tickets
C	30 tickets

Implementing Lottery Scheduling

```
int counter = 0;  
int winner = getrandom(0, totaltickets);  
node_t *current = head;  
while (current) {  
    counter += current->tickets;  
    if (counter > winner) break;  
    current = current->next;  
}  
// current gets to run
```

Who runs if **winner** is:

50
350
0



How to optimize this implementation (specifically reduce #walks)?

Lottery Scheduling (Misc)

A job can transfer its ticket to another

When is this useful?

Lottery Scheduling vs. RR

With equal tickets, each process is equally likely to be picked as winner and thus use CPU

RR also does the same...

What's the difference?

Linux CFS

Completely fair scheduler - used in Linux

Scheduling efficiency is a key goal – Google data center studies show that 5% CPU time spent in making scheduling decisions!

CFS high-level: fairly divides CPU evenly among all processes

Tension:

If need to decide too often → more fair but lots of overhead

If decide too infrequently → little overhead but less fair

Linux CFS - control parameters

Sched_lat – intuitively, sched_lat is the target interval over which CFS wants every runnable task to receive its fair CPU share, 48 ms is typical value

Vruntime (virtual runtime) – intuitively, how much CPU time actually consumed

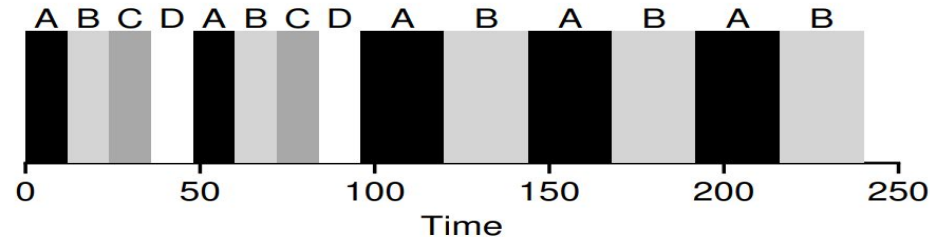
Time slice = $48 / \text{Num process}$ (dynamically decides quantum)

Example: with 4 processes, each gets 12 ms.

Pick the one with the lowest vruntime (intuition: has gotten less share)

If too many processes, then too much scheduling overhead, use a min_granularity control parameter (which is usually 6 ms)

Example: with 10 processes, each will get 6 ms



Linux CFS More

UNIX pri: -20 (high) < *nice* <= +19 (low) translates to a weight

If more nice, least priority → too nice to be scheduled now

If less nice, get more priority to use CPU

Time slice and vruntime adjusted based on nice value. High priority process gets longer slice and its vruntime grows slowly than lower priority process.

Runnable jobs kept in a (balanced) red-black tree, ordered by vruntime

All operations are $O(\log n)$

- Picking job with lowest vruntime

- Adjusting vruntime of a job

CPU VIRTUALIZATION SUMMARY

Mechanism

- Process abstraction

- System call for protection

- Context switch to time-share the CPU

Policy

- Metrics: turnaround time, response time, proportional share

- Various policies: eg. MLFQ

- Linux scheduling

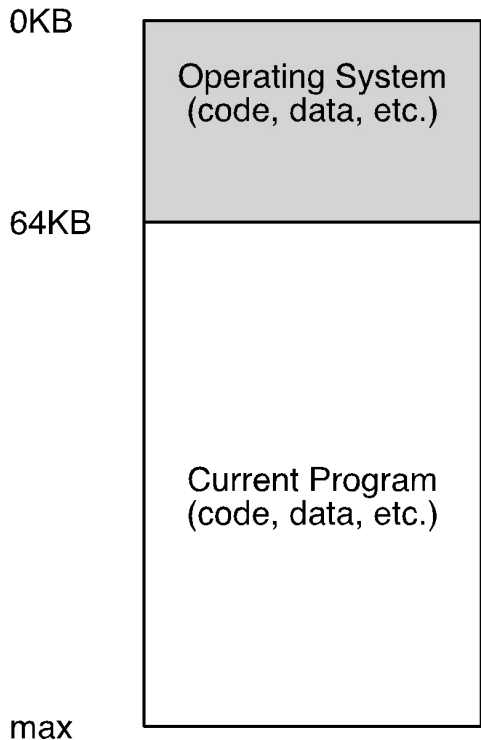
Virtualization

1st part of this course: Virtualization

First, OS virtualizes the CPU: illusion of private CPU for each process

Now, how OS virtualizes the memory: illusion of private memory for each process

BACK IN THE DAY...



Early systems did not virtualize memory

Uniprogramming: One process runs at a time

Disadvantages?

Only one process ready at a time

Process can destroy OS

MULTIPROGRAMMING GOALS

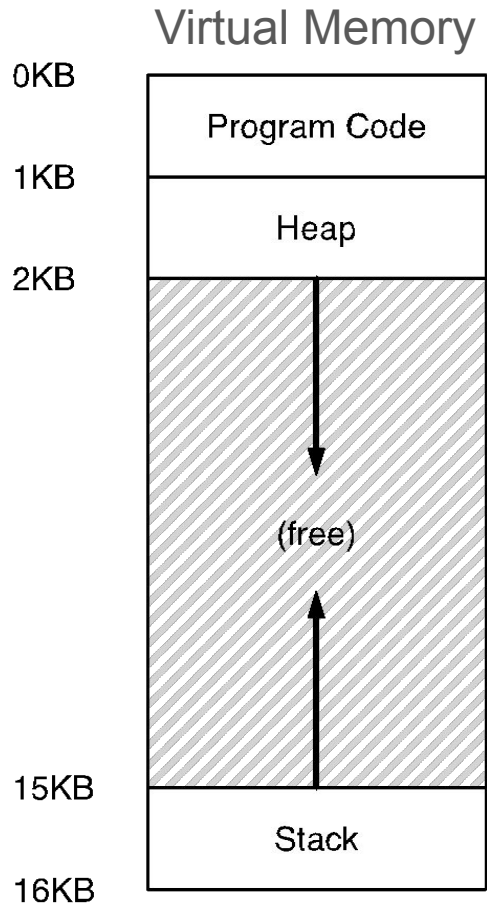
Transparency: Process is unaware of sharing physical memory

Protection: Cannot corrupt or read OS's or other process' memory

Efficiency: Do not waste memory (no fragmentation) or slow down processes

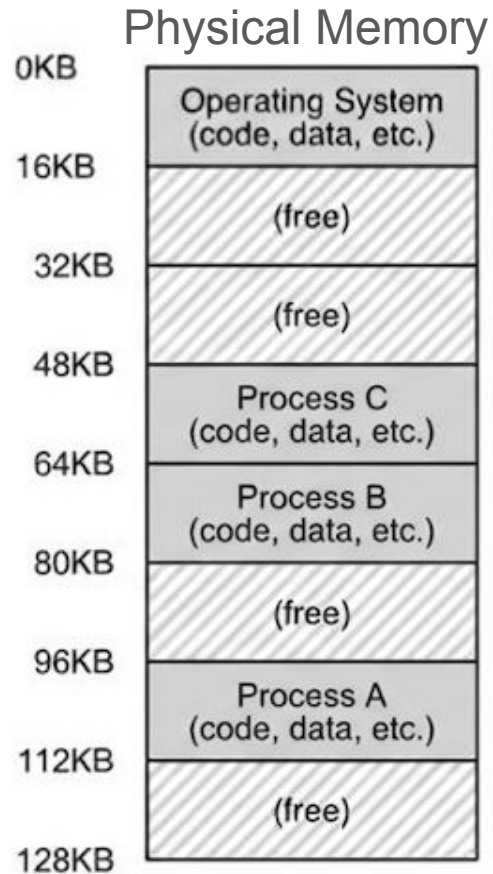
Sharing: Enable sharing between cooperating processes

ABSTRACTION: ADDRESS SPACE

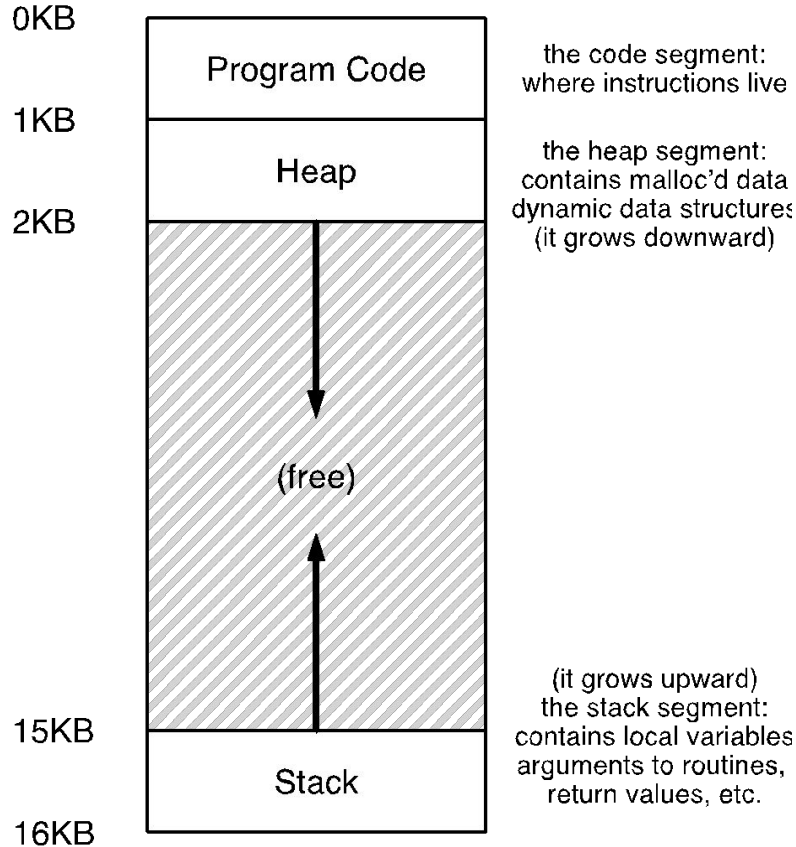


Virtual Address Space:
Each process has its
own address range

OS provides that
Illusion by mapping to
physical memory



WHAT IS IN ADDRESS SPACE?



Static: Code and global variables
Dynamic: Heap & Stack

Q: What if the process has multiple threads?

Why need dynamic memory?

Why do processes need dynamic allocation memory?

- Do not know how much memory needed at compile time

- Must do worst-case assumption and allocate more

Complex data structures: trees, graphs, lists – cannot say how much will be allocated

Recursive procedures: cannot say how many times procedure will be nested

Two types of dynamic allocation: heap and stack

STACK ORGANIZATION

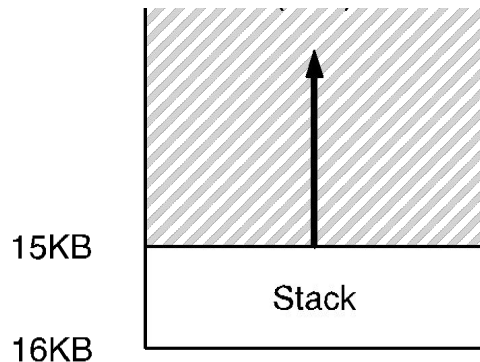
```
main()      : alloc(main)
  call A()   : alloc(A)
    call B() : alloc(B)
    exit B()  : free(B)
  call C()   : alloc(C)
  exit C()   : free(C)
exit A()     : free(A)
exit main()  : free(main);
```

SP demarcates allocated from free space

Call func: Increment pointer

Return from func: Decrement pointer

No fragmentation!



HEAP ORGANIZATION

Allocate from any random location: malloc(), new()

- Heap memory consists of allocated and free areas (holes)
- Order of mallocs and frees is unpredictable

Adv: works for all data structures

Disadv: Can result in fragmentation

How to allocate 20 (contiguous) bytes?

16 bytes

24 bytes

12bytes

16 bytes



What is OS's role in managing the heap?

OS gives out big chunks of free memory & library manages the individual allocations and frees

Possible locations:
static data/code, stack, heap

```
int x;  
int main(int argc, char *argv[]) {  
    int y;  
    int* z = malloc(sizeof(int));  
}
```

Address	Location
x	
main	
y	
z	
*z	

MEMORY ACCESS

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(int argc, char *argv[]) {
    int x;
    x = x + 3;
}
```

```
0x10:  movl    0x8(%rbp), %edi
0x13:  addl    $0x3, %edi
0x16:  movl    %edi, 0x8(%rbp)
```

%rbp is the frame pointer:
points to base of current stack frame

MEMORY ACCESS

Initial %rip = 0x10

%rbp = 0x200

➡ 0x10: movl 0x8(%rbp), %edi
0x13: addl \$0x3, %edi
0x16: movl %edi, 0x8(%rbp)

%rbp is the frame pointer:

points to base of current stack frame

%rip is instruction pointer (or program counter)

How many memory accesses?

To what addresses?

Chat with neighbors for 2 mins.

MEMORY ACCESS

Initial %rip = 0x10

%rbp = 0x200

➔
0x10: movl 0x8(%rbp), %edi
0x18: addl \$0x3, %edi
0x20: movl %edi, 0x8(%rbp)

%rbp is the frame pointer:
points to base of current stack frame

%rip is instruction pointer (or program counter)

5 memory accesses

load instruction at addr 0x10

Exec:

load from addr 0x208

load instruction at addr 0x13

Exec:

no memory access

load instruction at addr 0x16

Exec:

store to addr 0x208

HOW TO VIRTUALIZE MEMORY

Problem: How to run multiple processes simultaneously?

Compiler “hardcodes” the VM addresses into the binaries it generates

How to avoid collisions?

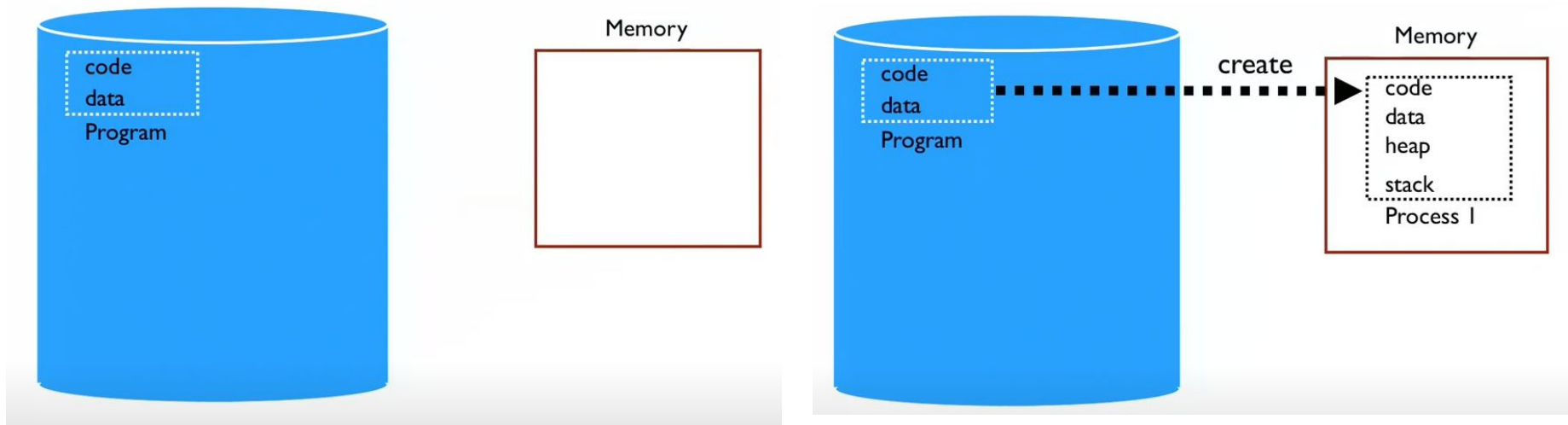
Possible Solutions for Mechanisms (in today’s class):

Time Sharing, Static Relocation, Base, Base+Bounds

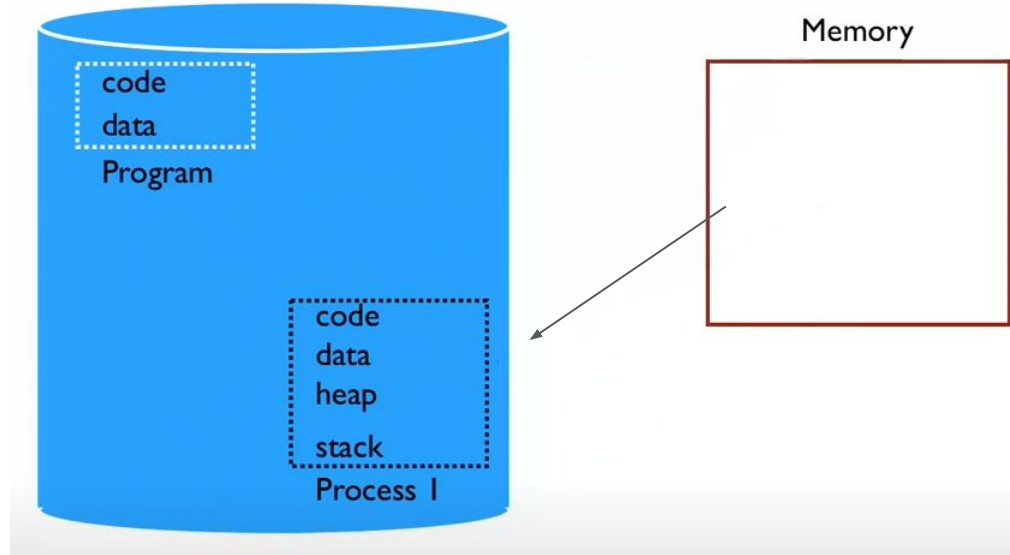
Unrealistic assumptions:

1. Each process gets the same sized VM address space, which is smaller than the physical memory on the machine
2. VM address space can be mapped contiguously in physical memory

1. TIME SHARE MEMORY



Time Sharing Memory



PROBLEMS WITH TIME SHARING?

Ridiculously poor performance

Better Alternative: space sharing

- Divide physical memory across processes

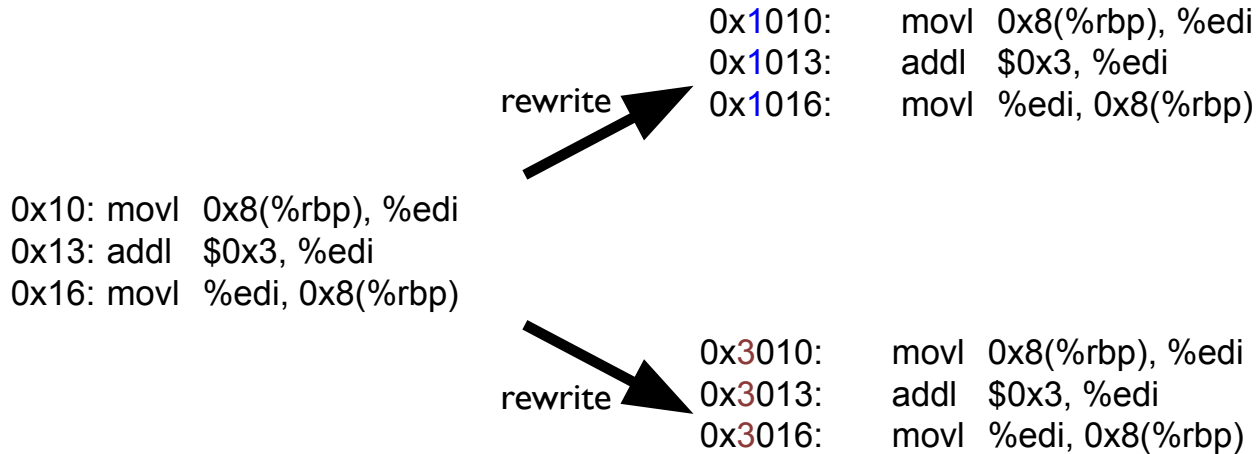
- Solutions discussed henceforth all use space sharing

LOADER: STATIC RELOCATION

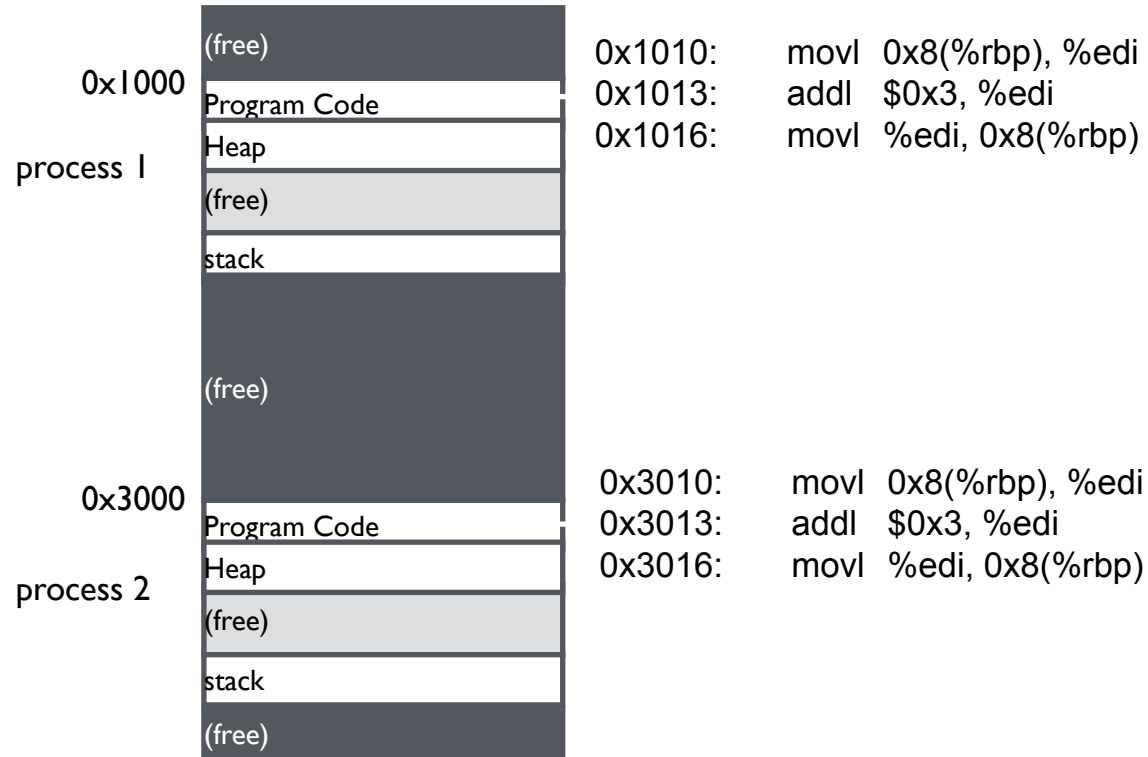
OS rewrites each program before loading it into memory (as a process)

Each rewrite for a process uses different addresses and pointers

Need to change jumps, loads of static data

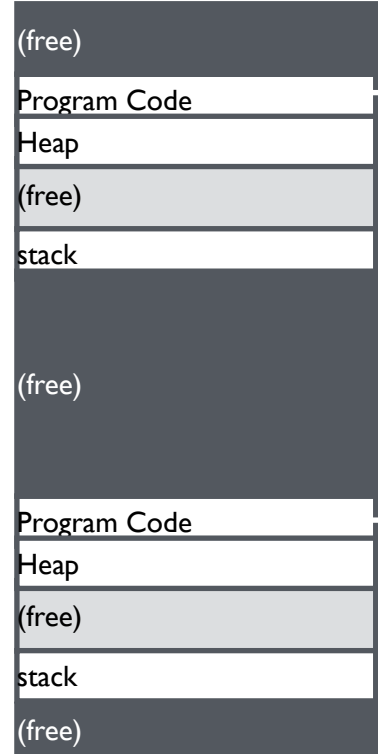


Static Layout in Memory



Static Relocation: Disadvantages

Chat for a minute with neighbors...



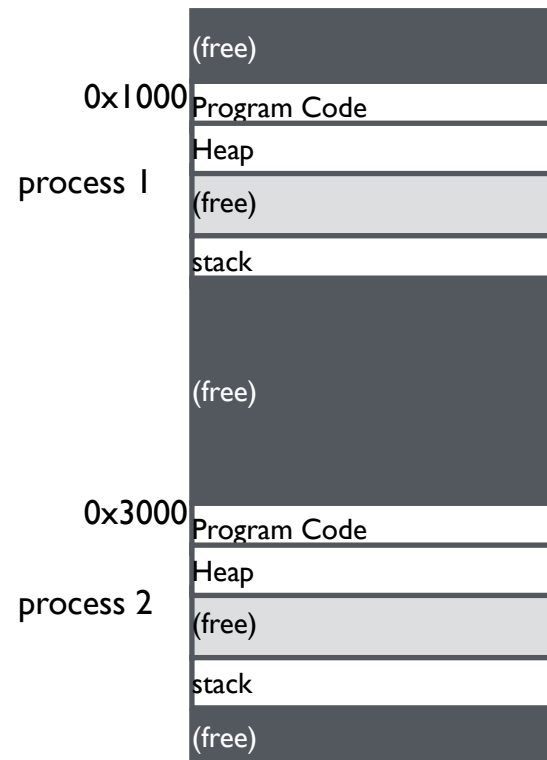
Static Relocation: Disadvantages

Cannot move address space after it has been placed

No protection

- Process can destroy OS or other processes (corrupt)
- No privacy (can read other process's memory)

Ex: `movl 0x2100, %eax` → `movl 0x3100, %eax`



3) Dynamic Relocation

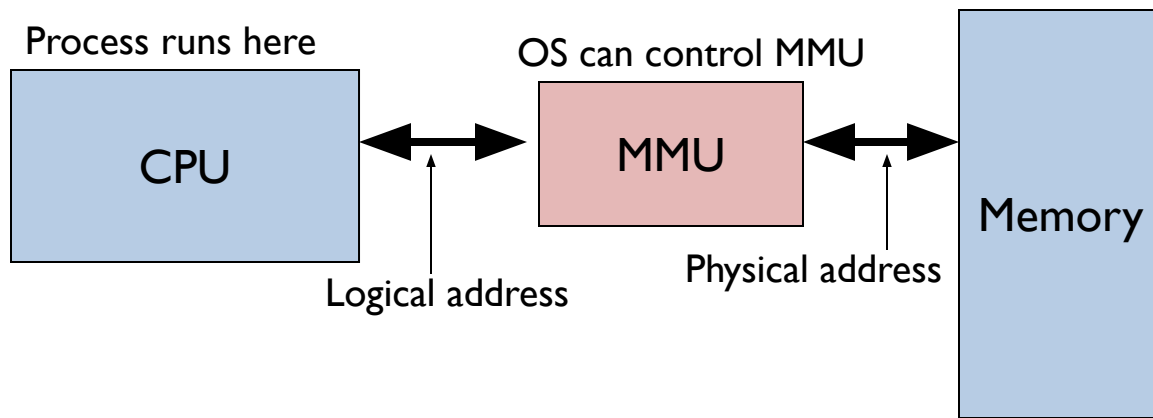
Goal: Dynamically relocate to different places, and protect processes from one another

Requires hardware support (so far OS was trying to virtualize alone!)

- Memory Management Unit (MMU)

MMU dynamically changes process address at every memory reference

- Process generates **logical** or virtual addresses (in their address space)
- Memory hardware uses **physical** or **real** addresses



HW Support for Dynamic Relocation

Privileged (protected, kernel) mode: OS runs

- When enter OS (trap, system calls, interrupts, exceptions)
- Allows certain instructions to be executed
(Can manipulate contents of MMU)
- Allows OS to access all of physical memory

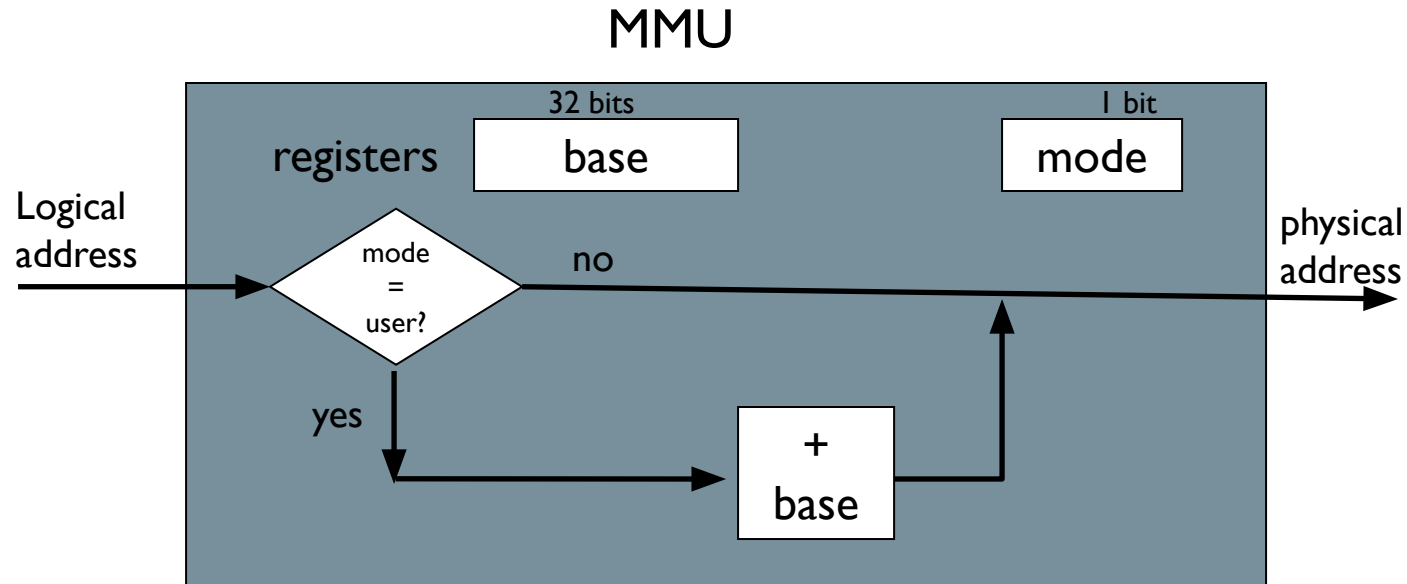
User mode: User processes run

- Perform translation of logical address to physical address

Implementation of Dynamic Relocation: BASE REG

Translation on every memory access of user process

MMU adds base register to logical address to form physical address



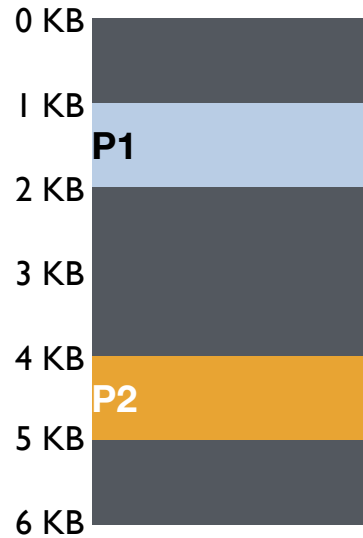
DYNAMIC RELOCATION WITH BASE REGISTER

Translate virtual addresses to physical by adding a fixed offset each time.

Store offset in base register

Each process has different value in base register

Dynamic relocation by changing value of base register



Virtual

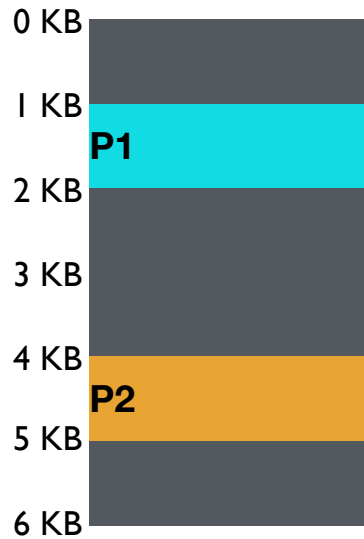
P1: load 100, R1

P2: load 100, R1

P2: load 1000, R1

P1: load 1000, R1

VISUAL Example of DYNAMIC RELOCATION:
BASE REGISTER



Virtual

P1: load 100, R1

P2: load 100, R1

P2: load 1000, R1

P1: load 1000, R1

Physical

load 1124, R1

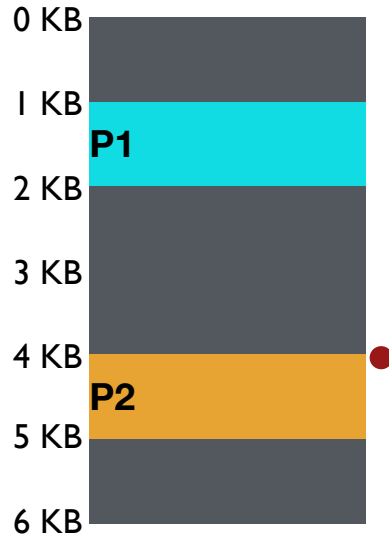
load 4196, R1

load 5096, R1

load 2024, R1

Can P1 hurt P2?

How well does dynamic relocation do with base register for protection?



Virtual	Physical
P1: load 100, R1	load 1124, R1
P2: load 100, R1	load 4196, R1
P2: load 1000, R1	load 5096, R1
P1: load 100, R1	load 2024, R1
P1: store 3072, R1	store 4096, R1 (3072 + 1024)

How well does dynamic relocation do with base register for protection?

DYNAMIC with BASE+BOUNDS

Idea: limit the address space with a bounds register

Base Reg: smallest physical addr (or starting location)

Bounds Reg: size of this process's virtual address space

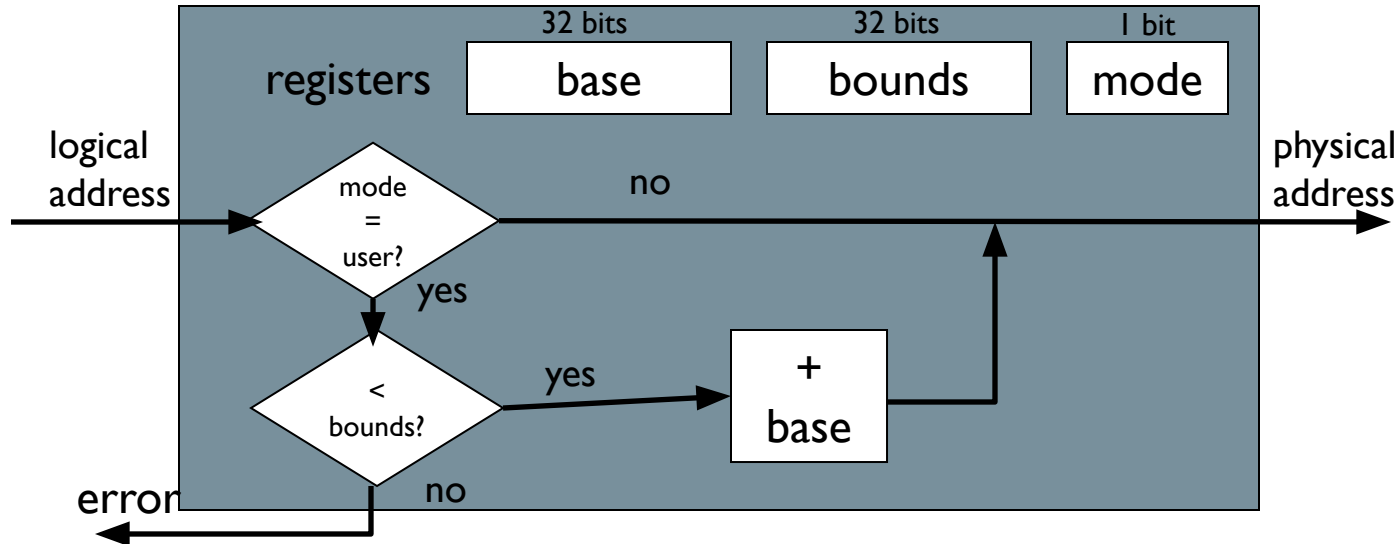
- The largest physical address ($\text{base} + \text{size}$)

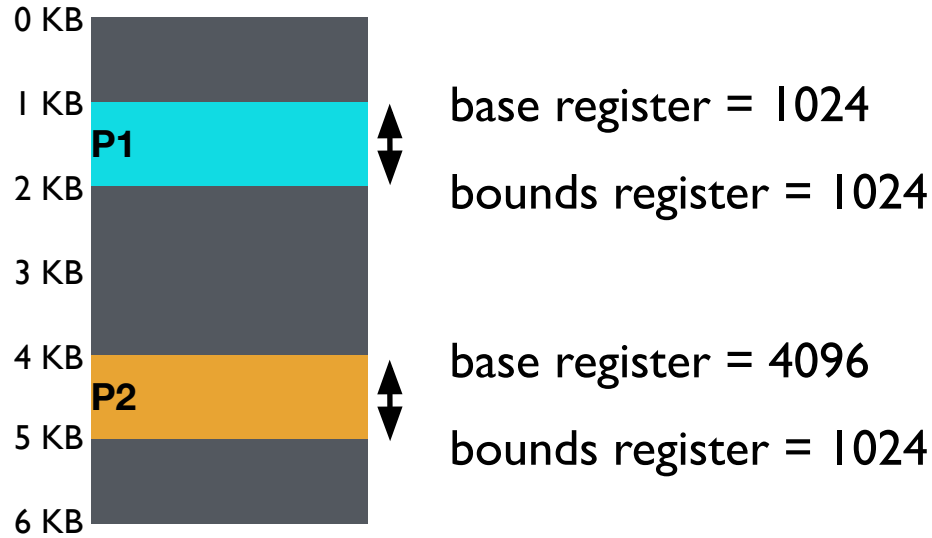
OS can kill process if it loads/stores beyond bounds

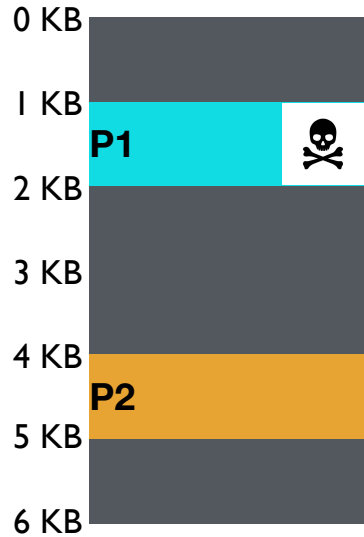
IMPLEMENTATION OF BASE+BOUNDS

Translate on every memory access in user mode

- MMU compares VM address to bounds register
If VM address is greater, then errors
- Else, MMU adds base reg to VM address => physical address







Virtual

P1: load 100, R1

P2: load 100, R1

P2: load 1000, R1

P1: load 100, R1

P1: store 3072, R1

Physical

load 1124, R1

load 4196, R1

load 5196, R1

load 2024, R1

Interrupt OS!

Can P1 hurt P2?

HW SUPPORT FOR DYNAMIC RELOCATION

Restricted (user) mode: MMU

- performs translation of logical address to physical address

Privileged (kernel) mode: OS can

- To manipulate contents of MMU
- Needed during context switching: load new base+bounds regs
- Allows OS to access all of physical memory

Managing Processes with Base and Bounds

Context-switch: Add base and bounds registers to PCB

Steps

- Change to privileged mode
- Save base and bounds registers of old process
- Load base and bounds registers of new process
- Change to user mode and jump to new process

Protection requirement

- User process cannot change base and bounds registers
- User process cannot change to privileged mode

Base and Bounds Advantages

Provides protection (both read and write) across address spaces

Supports dynamic relocation

- Can place process at different locations initially and also move address spaces

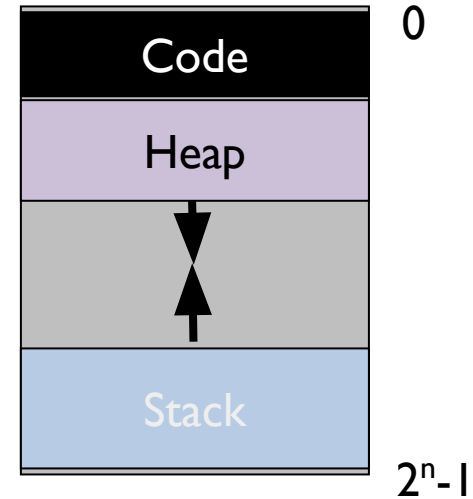
Simple, inexpensive implementation: Few registers, little logic in MMU

Fast: Add and compare in parallel

Base and Bounds DISADVANTAGES

Disadvantages

- Each process must be allocated contiguously in physical memory
Must allocate memory that may not be used by process
- No partial sharing: Cannot share parts of address space



Questions

1. Does the base register contain physical or virtual address?
2. Who converts VA to PA based on base register? Process? OS? HW?
3. Can the OS do the address translation? What is the downside?
4. Who modifies contents of base register?
5. What happens on a context switch to the base register?
6. Are there cases where the base register is not changed on switch?

NEXT LECTURES...

Relax assumptions:

1. VM address space $<$ physical memory address space
2. VM address space is mapped contiguously in physical memory

Segmentation

Paging

Segmentation + Paging

Multi-level Page Tables

TLBs